Agentic AI Foundation

AGNTCon + MCPCon China

Where the **agentic stack** is **being built**.

From Token Explosion to Semantic Trajectories

Optimizing Agent Execution in DeerFlow

Willem Jiang · DeerFlow Maintainer
Nan Gao· DeerFlow Contributor

Meet DeerFlow

OPEN-SOURCE SUPER AGENT

From an open-ended goal to a researched, cited deliverable.

A long-horizon, tool-using agent framework with orchestration, state, and recovery built into its Harness.

USER GOAL
Research, build, or automate

→

AGENT HARNESS
- LLM orchestration
- Tools & middleware
- State & checkpoints

→

DELIVERABLE
Report, code, or artifact

Harness decisions become token cost, latency, and reliability.

Deep Research is one of DeerFlow's long-horizon workflows.

How Deep Research Loops

A Harness coordinates models, tools, state, and checkpoints—repeatedly.

01
Plan
Research questions

02
Search
Sources & evidence

03
Evaluate
Relevance & gaps

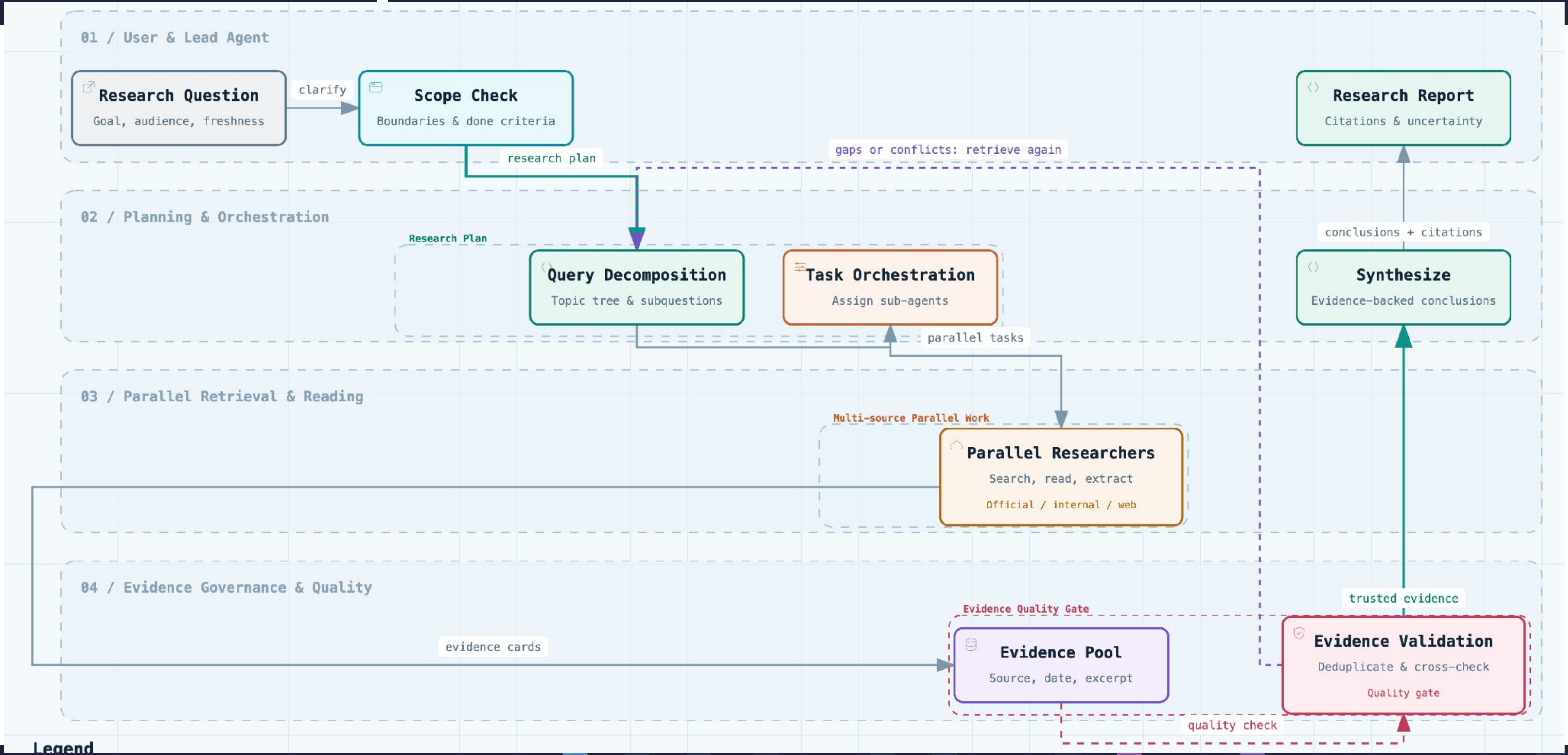
04
Refine
Next direction

05
Synthesize
Final report

↶ **Refine and continue until evidence is sufficient**

Every iteration adds state for the next model call.

How Deep Research Works

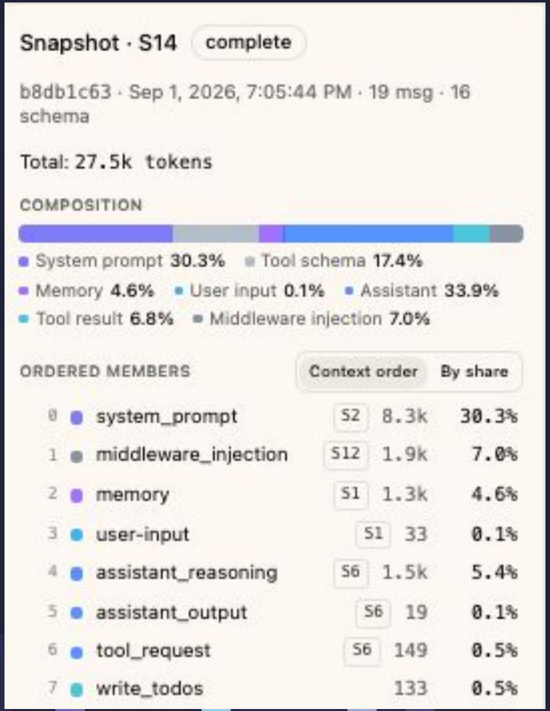


Tool Calls Stay in Context

SYSTEM PROMPT + USER INPUTS

PREVIOUS MODEL OUTPUTS
+ TOOL REQUESTS AND RESPONSES
+ ERRORS AND RETRY HISTORY

A large payload can be paid for again and again.



What Logs Fail to Show

CONVENTIONAL LOGS

What happened?

- LLM call completed
- Tool call failed
- Retry started
- Request took longer

MODEL-VISIBLE CONTEXT

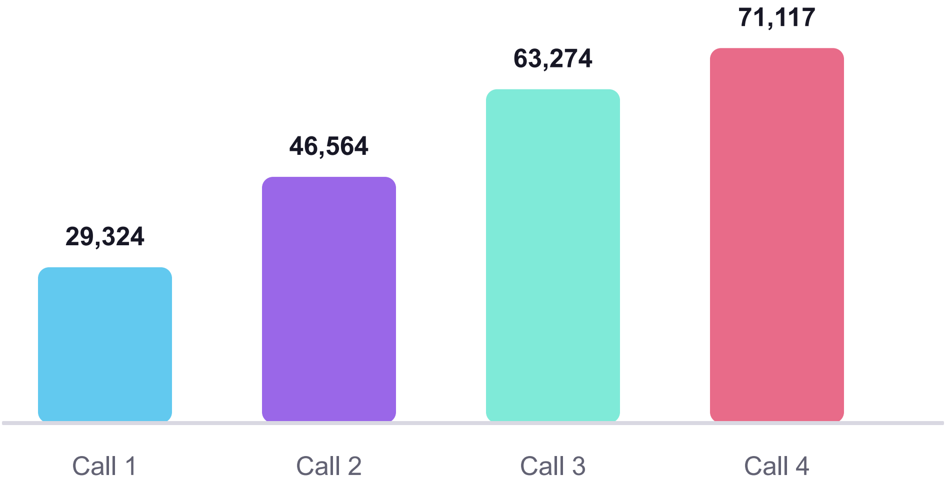
Why did the cost grow?

- Which payload remained?
- How much context did it occupy?
- Why did the agent retry?
- **Which event caused later cost?**

Events without causality do not explain execution cost.

A Token Spiral in DeerFlow

Deep Research task: collect the latest European football events



29K → 71K

input tokens across consecutive calls

Same run. Growing context.
Increasing cost.

Source: DeerFlow GitHub Issue #3114

How One Failure Multiplies

Agentic AI Foundation
AGNTCon + MCPCon
China

Large tool arguments	→	Tool call fails	→	Error echoes payload	→	State persists it	→	Retry resends it

≈24K characters echoed by one error path

A tool failure became a context-management failure.

Runtime Data to Belief

RUNTIME RAW DATA

01 RUNTIME PROBES

Lifecycle + model calls
Tool inputs / outputs
Context + usage

02 SERIALIZE & LINK

Filter known secrets
Build blocks + snapshots
Attach IDs + causal links

03 OBSERVATION

Typed event envelope
Who / when / subject
Payload + provenance

04 COLLECT & WRITE

Bounded queue, fail-open
Async write + deduplicate
Commit events + jobs

Recorded facts support each judgment

05 PROJECTORS

Tasks, Steps + ToolCalls
Context, lineage + usage
Trace evidence windows

06 ASSESS / ENRICH

Rules or optional LLM
Read evidence windows
Propose judgments

rules

07 BELIEF ASSERTIONS

Value + linked evidence
Assessor version + config
Authority + evidence time

08 RESOLVE BELIEF

Choose by authority + time
Retain other assertions
Unknown / unassessed

Recording a label proves the judgment was made. Its content still carries its own fidelity.

Trajectories Explain What, Not Why

MAINSTREAM TRAJECTORIES

Structured raw data

Span trees · calls · latency · inputs and outputs · checkpoints

They answer: what happened, how long, how much.

THE QUESTION THEY CANNOT ANSWER

Why this decision, at this moment?

Renaming web_search to “information gathering” is still a description of the action, not a reason.

NO STATE

The context the agent saw is not recorded

NO GAP

What was missing is never named

HINDSIGHT

Later outcomes leak into the label

A Why Is a Decision Relative to a Gap

PHASE

What stage is it in?

plan · act · verify · present · recover

GAP

What is missing right now?

information · confirmation · artifact · permission · decision

STANCE

What does this step do about it?

acquire · verify · transform · deliver · repair · decide · abandon

EXAMPLE · THE 7TH WEB SEARCH

phase act · information gaps closed · one decision gap still open · stance acquire. It keeps gathering when it should decide.

Deliberation: Label the Gap and the Stance

DECISION SIDE

decision_kind open | forced
phase plan · act · verify · present · recover
goal the sub-goal this step serves
gap_before [{type, target}] — what is missing
action_role → aims_at the stance and which gap it is for
decision_basis must point into the window

OUTCOME SIDE

evidence_delta gaps closed · gaps opened
environment_note tool result quality: ok · degraded · failed · unknown

Revealed only after the decision fields are committed.

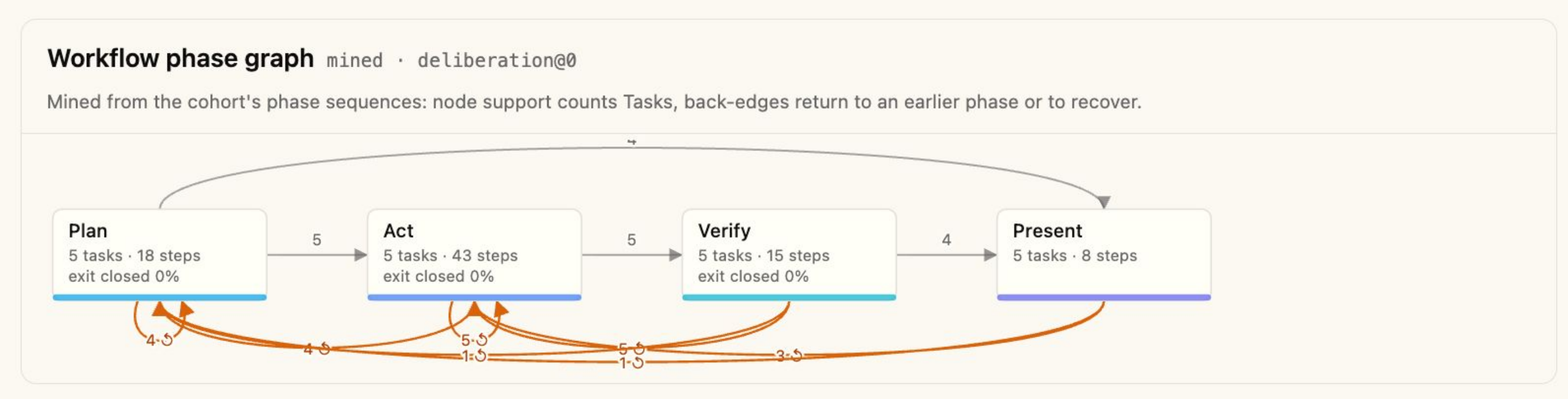
RULE ASSESSOR

tool classes · lineage edges · execution states — zero cost, never guesses

LLM ANNOTATOR

sees exactly the agent's window · two-stage prompt · replay identity by config hash

why(step) = phase · gap_before · action_role → aims_at · decision_basis


Workflow phase graph mined · deliberation@0
Mined from the cohort's phase sequences: node support counts Tasks, back-edges return to an earlier phase or to recover.
Plan
5 tasks · 18 steps
exit closed 0%
5
Act
5 tasks · 43 steps
exit closed 0%
5
Verify
5 tasks · 15 steps
exit closed 0%
4
Present
5 tasks · 8 steps
4
4
5
1
5
1
3

Workflow vs. Behavior Pattern

WORKFLOW

Imposed by the task

A phase graph with entry and exit conditions on gaps. Any agent on this task has to pass through it.

stable across releases · varies by domain

web_search	domain X	domain Y
release A	4	2
release B	4	2

BEHAVIOR PATTERN

Imposed by the agent

A conditional policy: given phase and gap, what it tends to do, how many times, when it stops.

stable across domains · varies by release

web_search	domain X	domain Y
release A	2	2
release B	4	4

Workflow is the coordinate system; behavior is the trajectory statistics inside it.

From Policy Table to Optimization Target

P(ACTION | GAP) · OPEN STEPS ONLY

	acquire	verify	decide	deliver
information	**0.83**	0.09	0.02	0.06
confirmation	0.12	**0.71**	0.10	0.07
decision	**0.41**	0.08	**0.38**	0.13

TRACE → DELIBERATION → PATTERN

CONTRAST WITH SUCCESS

Same cell as a predicate: success rate with vs. without

lift ≥ 0.2 · n ≥ 3 on each side · quality beliefs, not impressions

OPTIMIZATION TARGET

threshold_miscalibration → stop criterion
workflow_gap → skill scaffold
repair_strategy → tool description

Solidify the Pattern into the Harness

WHAT GETS SOLIDIFIED

A predicate and a statistic

workflow phase graph with gap entry / exit rules

behavior P(stance | phase, gap) and loop limits

target contrast plus an actuator hint

re-evaluable on any cohort · never a prompt, never a transcript

FOUR ACTUATORS

Target kind → harness knob

skill scaffold phases and exit conditions in SKILL.md

stop criterion k same-stance steps, no closure → move on

tool description remap gap → stance

delegation verify · decide go to the stronger model

THE INTERFACE

A declared ledger

The cheaper model writes its gaps as todos; **satisfied** is its own completion mark.

Exit predicates become mechanically checkable by the harness.

no LLM judgement at runtime · the rule assessor reads it

CLOSED LOOP · THE SAME MACHINERY

Mine the frontier cohort → **Register a normative workflow** → **New release: cheap model + scaffold** → **Run a cohort, mine again** → **Diff vs. normative · contrast**

Structure and thresholds transfer. Judgement does not: keep it in the ledger, or delegate it.

Replay Makes Bugs Reproducible

ORIGINAL TRAJECTORY

Checkpoint → Tool failure → Retry → Context growth

AFTER HARNESS CHANGE

Checkpoint → Bounded error → Recovery → Continue

Select → Restore → Inspect → Re-run → Compare

Replay controls the starting state—not model determinism.

Three Lessons for Reliable Agents

01 Task success is not execution efficiency.

02 Context is a shared, accumulating resource.

03 Intent-aware trajectories turn opaque behavior into engineering evidence.

Agentic AI Foundation

AGNTCon + MCPCon China

Where the **agentic stack** is **being built**.

Questions?

Visible · Replayable · Optimizable

Willem Jiang · Nan Gao

https://github.com/bytedance/deer-flow